

Construyendo las Damas en Android



De la Lógica pura a la Pantalla: Una guía visual de Kotlin y Jetpack Compose para principiantes.

La Regla de Oro: Separar la Lógica de la Interfaz

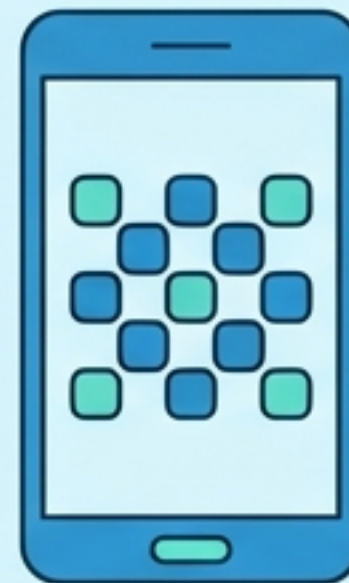
El Cerebro (Kotlin)



La Lógica Pura: Aquí viven las reglas, los turnos y la validación de movimientos. El código de esta sección no sabe qué es un botón ni una pantalla; podría jugarse en una consola de texto.

Mantener estas dos áreas separadas es el secreto de una arquitectura limpia en Android.

El Rostro (Jetpack Compose)



La Vista: Su única función es dibujar lo que el cerebro le dicta y capturar los toques del usuario. No toma decisiones sobre el juego.

El Punto de Partida: Donde Android conoce a Compose

```
class MainActivity : ComponentActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            PantallaDamas()  
        }  
    }  
}
```

La Ventana del Sistema:

Esta es la base que Android utiliza para lanzar la aplicación. Es el marco del cuadro.

El Puente Mágico:

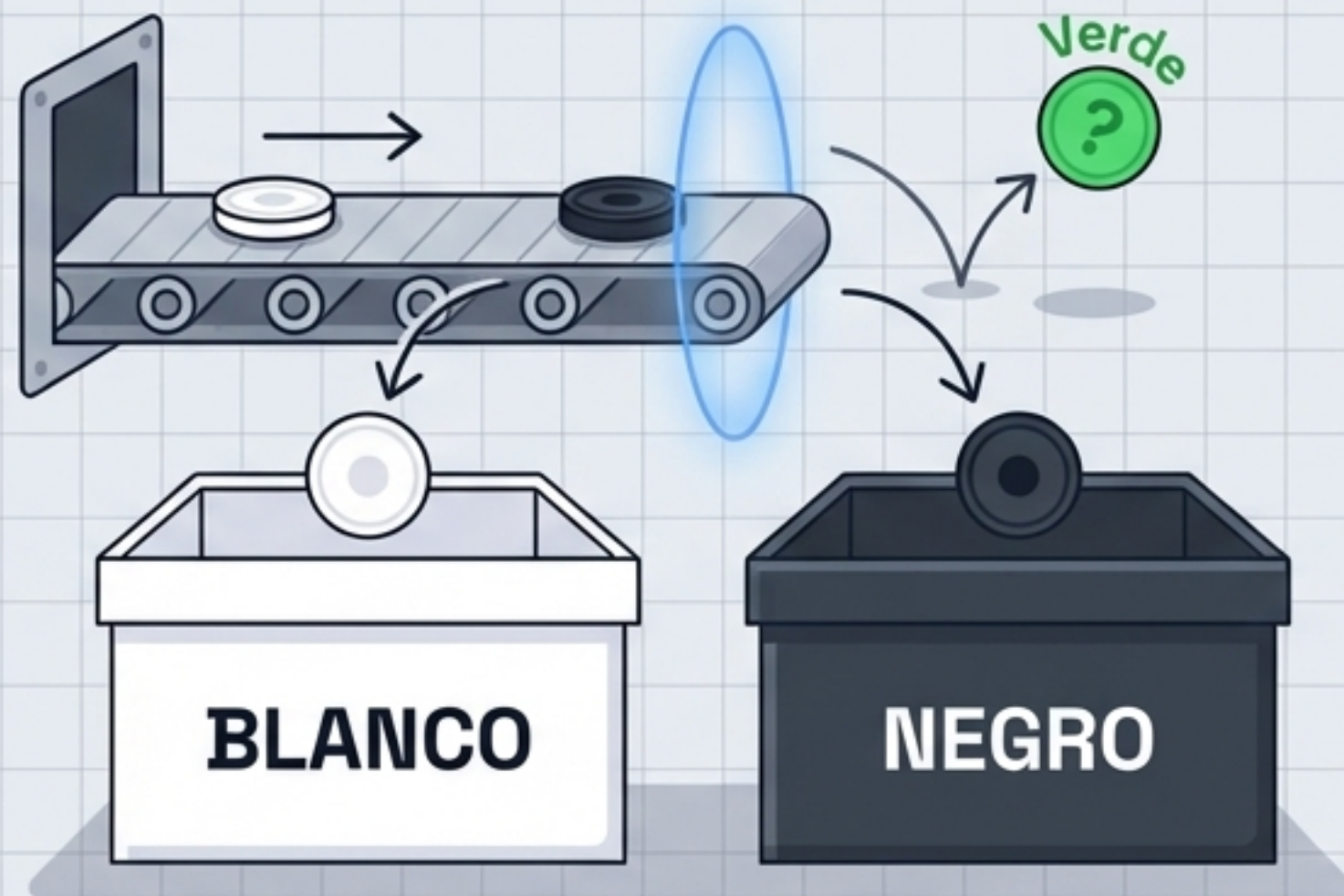
Este bloque es crucial. Aquí le decimos a Android: A partir de este punto, Jetpack Compose toma el control total de dibujar la pantalla.

Nuestra Interfaz:

La llamada a la función que dibujará nuestro tablero.

Opciones Fijas y Seguras: enum class

- **¿Qué es un Enum?** Es una herramienta de Kotlin para definir una lista cerrada de opciones.
- En nuestro juego, solo existen dos opciones válidas: BLANCO y NEGRO. No puede haber un jugador Verde o un error tipográfico. Los Enums obligan al código a ser estricto.



```
enum class Jugador(val texto: String) {  
    BLANCO("Blancas 🏳️"),  
    NEGRO("Ordenador 🖤") // En esta versión, las negras siempre son el ordenador  
}
```


La Anatomía de las Piezas: Relacionando Datos

	Equipo Blanco	Equipo Negro
Vacío	Símbolo: "" Pertenece a: null	
Peones	Símbolo: ○ <code>Jugador.BLANCO</code>	Símbolo: ● <code>Jugador.NEGRO</code>
Damas (Coronadas)	Símbolo: ♔ <code>Jugador.BLANCO</code>	Símbolo: ♚ <code>Jugador.NEGRO</code>



Un Enum no solo es una etiqueta. Al agrupar el símbolo visual y el equipo al que pertenece dentro de enum `class TipoPieza`, le enseñamos a cada pieza quién es su dueño desde el principio, ahorrando decenas de líneas de código condicional más adelante.

Empaquetando la Información: data class

TICKET DE MOVIMIENTO

Origen

[filaOrigen : Int]

[columnaOrigen : Int]

Destino

[filaDestino : Int]

[columnaDestino : Int]

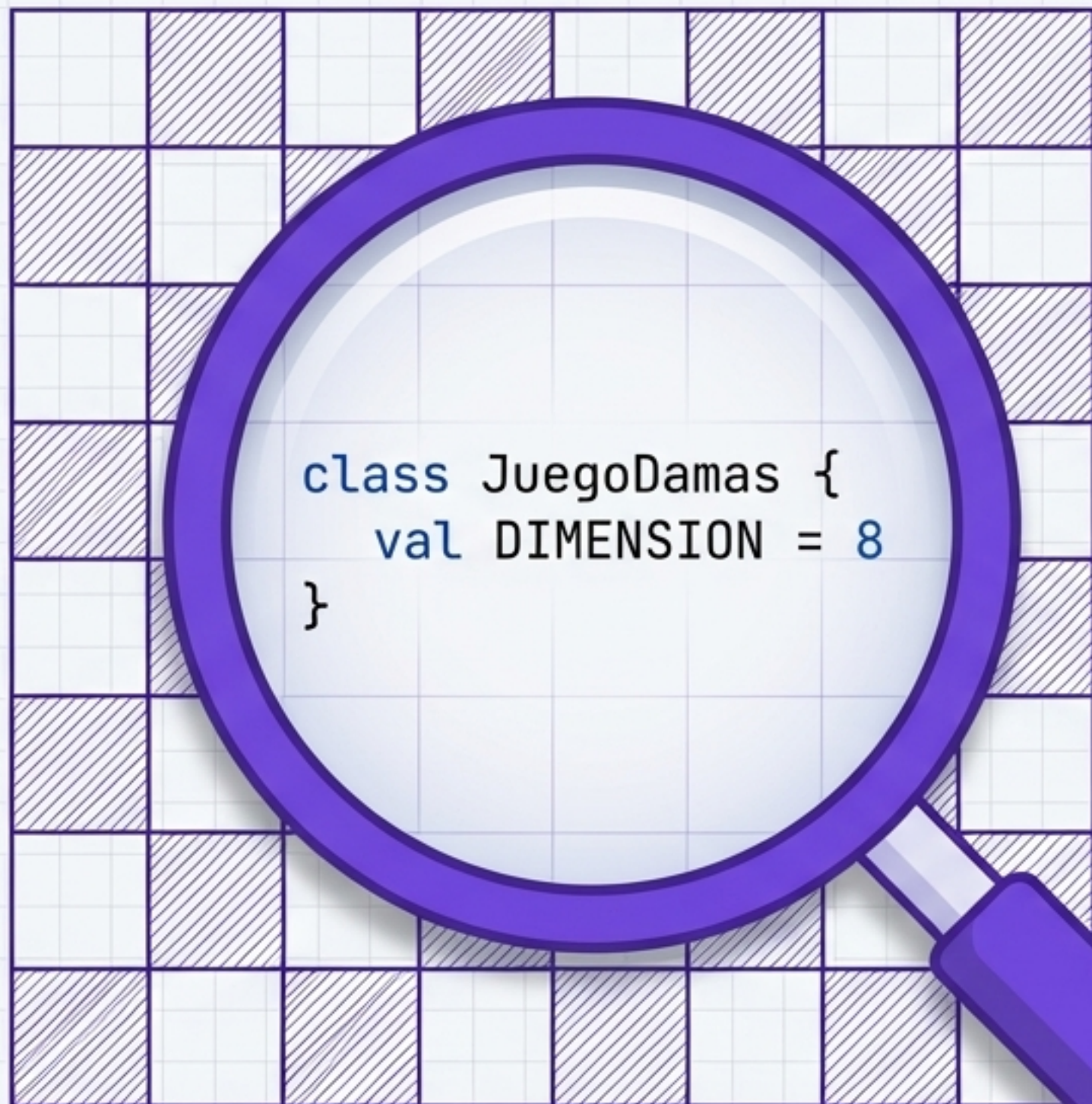
Alerta de Captura

[esCaptura : Boolean]



Una data class en Kotlin nace únicamente para **transportar información**. Cuando el ordenador analiza el tablero, anota todos sus movimientos posibles en tickets como este para luego decidir cuál es la mejor jugada. Son estructuras ligeras, ideales para mover datos de un lado a otro.

El Tablero de Comando: La clase JuegoDamas

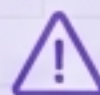


El Árbitro Invisible:

Esta clase principal contiene toda la lógica interna del juego.

Responsabilidades:

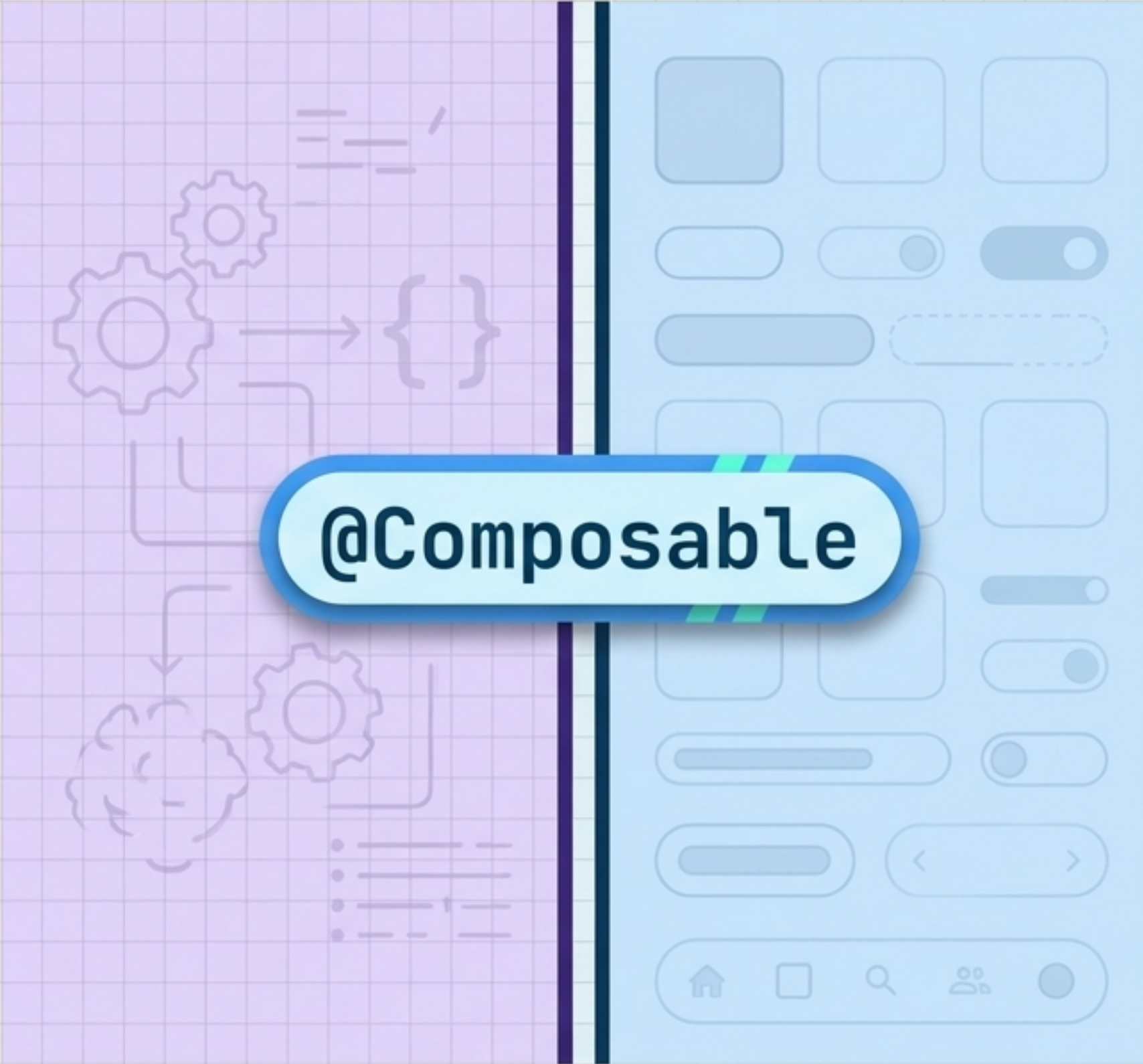
Aquí es donde se define el tamaño del tablero, donde se valida si un movimiento hacia adelante es legal, y donde se controlan los turnos de los jugadores.



Nota Arquitectónica:

Observa que esta clase está **completamente aislada**. No contiene ningún color, ningún botón, ni ninguna orden de dibujo. Es pura lógica de Kotlin.

Cruzando al Rostro: La Magia de @Composable



@Composable

Entramos a Jetpack Compose. Aquí definimos la Vista mediante la función `PantallaDamas()`.

¿Qué hace `@Composable`? Es una etiqueta (anotación) que convierte una función normal de Kotlin en un dibujante.

Le dice al sistema Android que esta función tiene el poder de emitir componentes visuales (cuadrados, textos, botones) directamente en la pantalla de un teléfono.

Protegiendo la Memoria: El concepto de remember

Recomposición:
Parpadeos
invisibles



Heavy, highly
secure iron
safe box



```
val motorJuego = remember {  
    JuegoDamas()  
}
```

Jetpack Compose redibuja la pantalla constantemente para reaccionar a los toques del usuario. Si no usamos **remember**, cada parpadeo de la pantalla crearía un tablero nuevo y la partida se reiniciaría desde cero.

La función **remember** es el ancla de seguridad que mantiene vivo el Cerebro entre cada redibujo de la pantalla.

La Anatomía Visual: Esculpiendo con Modifiers

Pintura

JetBrains Mono
`Modifier.background()`

Inter
Define si la casilla se dibuja clara u oscura.

Dimensiones

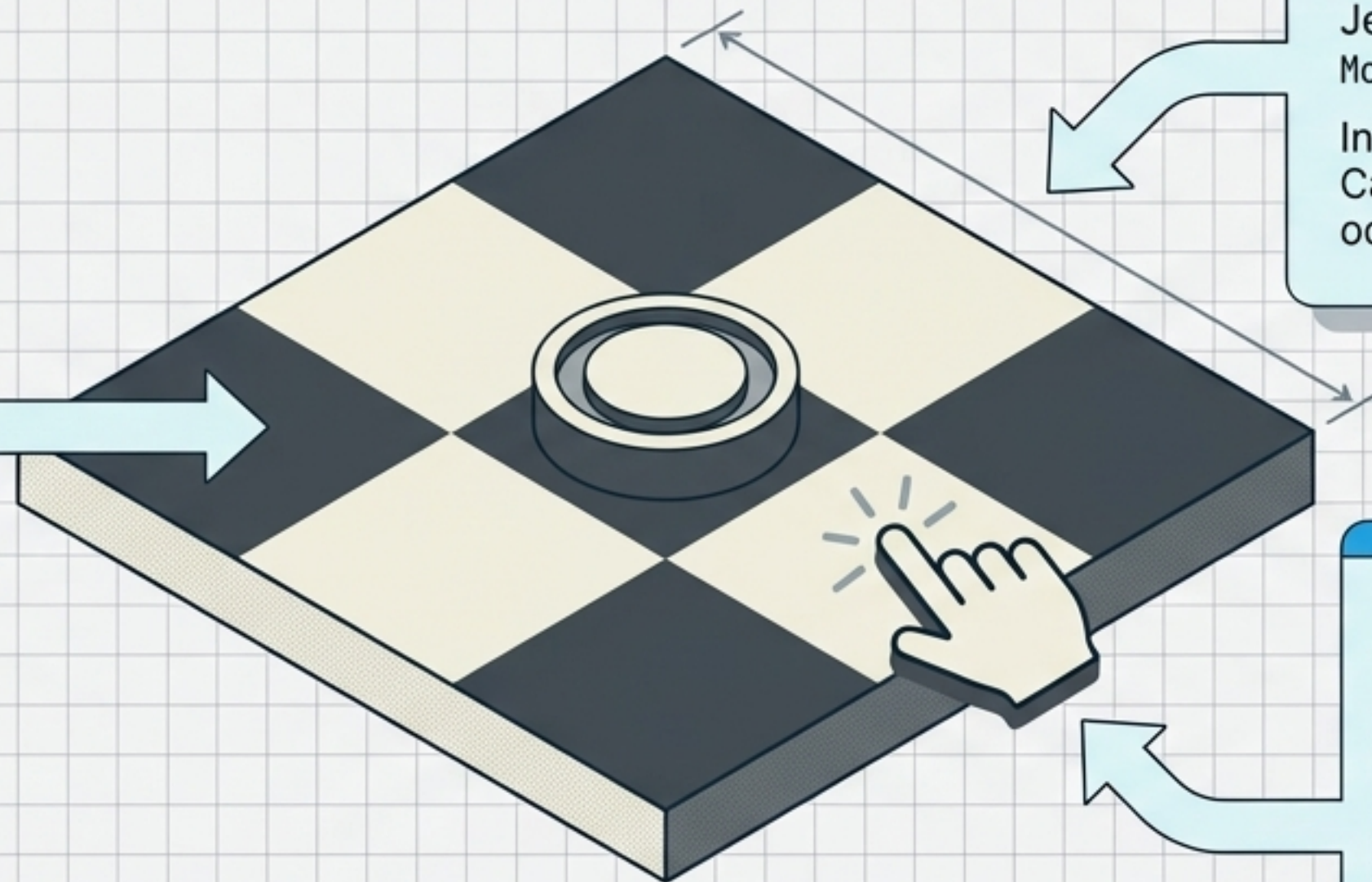
JetBrains Mono
`Modifier.fillMaxSize() / .size()`

Inter
Calcula el tamaño exacto que ocupará en la cuadrícula.

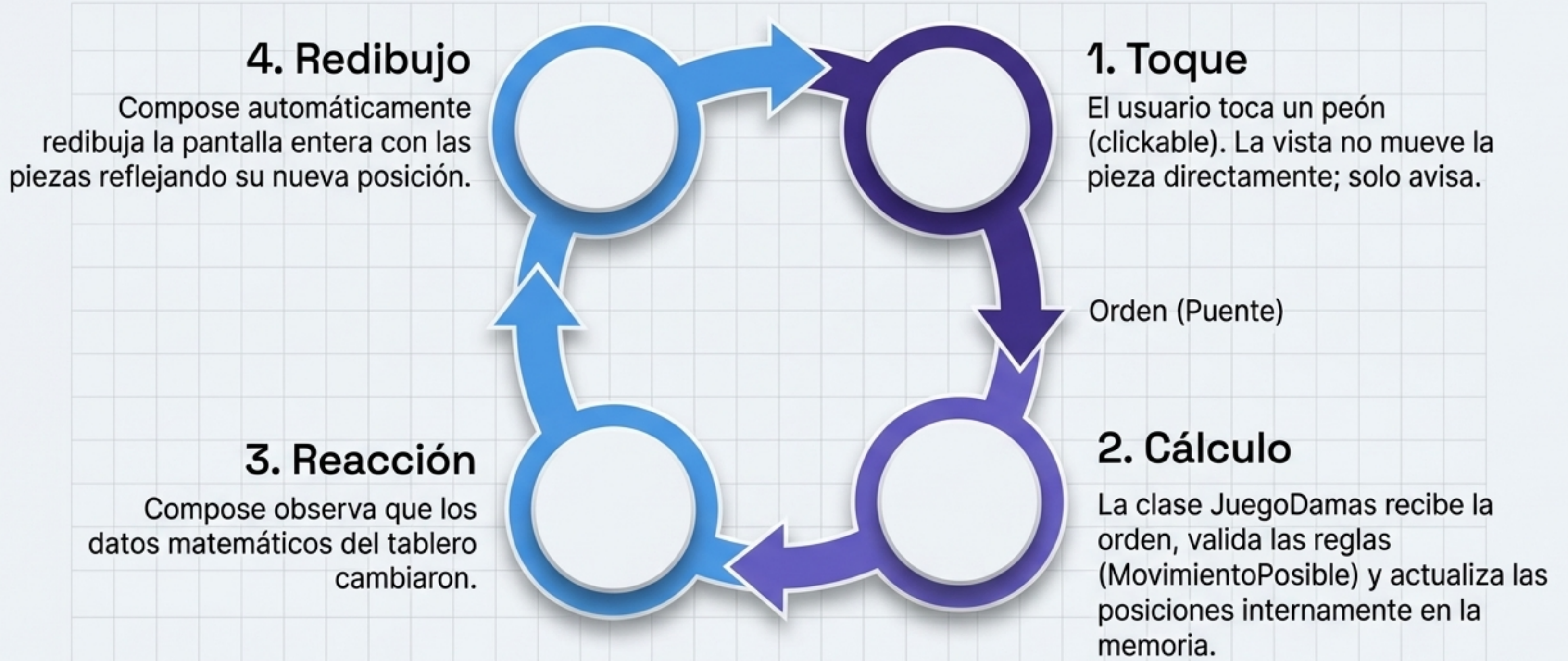
Sensores Táctiles

JetBrains Mono
`Modifier.clickable { ... }`

Inter
El cableado interactivo que detecta el toque del usuario y envía la señal de acción hacia el Cerebro.



La Coreografía Final: El Bucle Reactivo



La Vista nunca cambia sus propios datos; solo informa y obedece al Cerebro.

Hoja de Ruta: De un Vistazo



enum class

Define opciones fijas y cerradas (BLANCO, NEGRO). Da seguridad al código evitando valores inválidos.



data class

Estructuras ligeras creadas exclusivamente para transportar información (ej. coordenadas de origen y destino).



@Composable

La etiqueta mágica que transforma una función normal en un componente visual capaz de dibujarse en la pantalla de Android.



remember

Protege tus variables (como el motor del juego) para que sobrevivan intactas a las recomposiciones (redibujos) de la interfaz.